2017 13th IEEE Conference on Automation Science and Engineering (CASE)
Xi'an, China, August 20-23, 2017

Model-based Coverage Motion Planning for Industrial 3D Shape Inspection Applications

Wei Jing[1], Joseph Polden[2], Pey Yuen Tao[2], Chun Fan Goh[1], Wei Lin[2] and Kenji Shimada[1]

Abstract— **In this paper, we propose a novel planning method for the shape inspection applications with a sensor-equipped industrial manipulator. In the proposed approach, the view planning problem is formulated as a Set Covering Problem (SCP) and the path planning problem is formulated as a Travelling Salesman Problem (TSP). The combined SCP and TSP problems are then formulated as a sequencing problem, and solved simultaneously via Random-Key Genetic Algorithm (RKGA) in a one-step optimization process. With the proposed method, time-efficient and collision-free motion plan that satisfies the requirement of the robotic inspection tasks can be generated automatically. Computational studies were performed to verify the effectiveness of the proposed planning method. The method was found to outperform other approaches by generating robot paths that complete the inspection task with shorter execution time. The Overall time reductions were found up to 28.4% compared to the previous approaches. To further demonstrate the effectiveness of the proposed method in a real-world inspection task, an experimental test was also performed using an ABB industrial robotic system equipped with an on-board 3D scanner.**

I. Introduction

As a result of the rapid development of robotic technologies in manufacturing sectors, inspection of mechanical parts in production lines is now often performed automatically via robotic automation [1][2]. In many manufacturing cases, the final shape of the manufactured product may differ from its originally intended design due to the irregularities encountered during the manufacturing processes. In such cases, conducting 3D shape inspection will enable the quality assurance by quantifying the differences between a scanned model of the final product and a 3D CAD model of its intended design [3]. Since this task is of repetitive nature, particularly in the production line featuring large volumes, generating a time-efficient robot motion plan for the inspection task is of particular importance.

Using an industrial manipulator to conduct the inspection work requires two key tasks: First, a set of viewpoints that meet the required inspection goal(s) must be identified. The process of finding these viewpoints is called view planning [4][5][6]. The second task is to determine the optimal sequence of viewpoints to visit and the collision-free robot paths among them. This is usually formulated as a path planning problem [7][8].

In years past, the view planning problem has been studied intensively across a number of different applications [9][10]. View planning can be categorized as model-based view planning or non-model-based view planning, depending on whether the geometric information of the target object is available or not in advance [5]. For the non-model-based view planning [11], information gain [12] is usually used to select viewpoints in a Next-Best-View (NBV) [13] framework. In the shape inspection application discussed in this paper, the geometric model of the target is known in advance, thus the model-based view planning method is more suitable. For the model-based view planning [11][14], the "generate-test" methods [5][15] were usually applied. These methods formulated the view planning problem as a Set Covering Problem (SCP) by generating a large set of candidate viewpoints, from which a subset of suitable viewpoints was then selected via combinatorial optimization.

The overall motion planning process also requires the determination of a suitable sequence to visit each selected viewpoint. Additionally, collision-free robot paths must be planned to ensure that the robot can move safely about its workspace as it visits the generated viewpoints. The path planning problem in this context usually refers to finding the sequential robot paths with the minimum cost through a set of way-points in the robot task space, as described in the literature [7][8][16]. The Traveling Salesman Problem (TSP) is then formulated for the path planning problem in the industrial inspection application.

Traditionally, the SCP and TSP were formulated and solved separately for the inspection applications [17][11], which degrades the overall quality of the optimization results. More recently, traveling cost has been considered by researchers alongside viewing costs [18] in the optimization problem formulation, where Linear Programming and rounding was used to solve the problem, but the work mainly focused on Euclidean space and did not implement the method with real robot simulation or experiment. Later work of the same researchers [19] proposed algorithms that solve the two NP-hard problems separately. Researchers [10][20] also proposed a sampling-based redundant roadmap method for the large scale underwater inspection applications, the SCP and TSP are considered as a single combined optimization formulation, but several approximations are used and the two

[1]Wei Jing, Chun Fan Goh and Kenji Shimada are with Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, 15213, USA. {wj,chunfang}@andrew.cmu.edu, shimada@cmu.edu

[2]Joseph Polden, Pey Yuen Tao and Wei Lin are with Mechatronics Group, Singapore Institute of Manufacturing Technology, 71 Nanyang Drive, 638075 Singapore; and SIMTech-NUS Joint Laboratory on Industrial Robotics, Department of Mechanical Engineering, National University of Singapore, 117575 Singapore. {poldenjw, pytao, wlin}@simtech.a-star.edu.sg

This work is partially supported by the Collaborative Research Project under the SIMTech-NUS Joint Laboratory on Industrial Robotics, Ref: U13-R-024AU and the SERC Industrial Robotics Program Work Package 8, 122510008.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.

NP-hard problems are also solved sequentially.

In this paper, we propose a novel planning method for the industrial inspection applications, where a sensor-equipped industrial manipulator is used to perform a 3D shape inspection task. The proposed method first formulates the SCP and TSP individually for the given target object, then combines the two problems as a single sequencing optimization problem, and solves it simultaneously using Random-Key Genetic Algorithm (RKGA). The proposed method also generates the resultant trajectory as a ROS-compatible format that can be directly used for practical applications. The main contributions of this paper are:

- a novel sequencing optimization problem formulation and a RKGA-based approach for the combined SCP and TSP problems, which leads to better results;
- an encoding/decoding strategy of RKGA that handles the coverage and TSP constraints in the fitness evaluation of chromosome, which simplifies the optimization process;
- a combined SCP and TSP formulation for robotic inspection application with validation and benchmark of real-world robotic data; and
- a overall framework to generate efficient motion plan for the industrial shape inspection applications with robotic manipulator and 3D scanner.

II. Proposed Problem Formulation

For the robotic inspection task discussed in this paper, a 3D scanner is mounted on a 6 Degree-Of-Freedom (DOF) industrial manipulator and the target object is placed in front of the robot. The robot is required to begin the inspection process from its home position, travel to the planned viewpoints and take the required measurements with its on-board sensor; after all the required data is captured, the robot should return to its initial home position to prepare for the next inspection. The overall planning process is to find the set of required viewpoints together with the corresponding robot poses, the visiting sequence for the viewpoints, as well as collision-free robot paths between them, such that the overall cycle time required for the inspection task is minimized.

A. SCTSP Formulation

In this paper, we formulate the planning problem of the overall industrial inspection task in such a way that the SCP and TSP are solved simultaneously, which we term as the Set-Covering-Traveling-Salesman-Problem (SCTSP).

The problem is formulated in discrete space by sampling. The notations to form the optimization problem are listed as follows. The candidate viewpoint set $\mathbb{V}$ has n viewpoints. The surface patch set $\mathbb{S}$ has m patches of the target object. A undirected Graph $\mathbf{G}(\mathbb{V}, \mathbb{E})$ encodes the traveling costs information among the viewpoints. e_{ij} or $e(v_i, v_j) \in \mathbb{E}$ is the edge between v_i and v_j, and it represents the traveling cost between viewpoints v_i and v_j. A $n \times m$ binary visibility matrix $\mathbf{A}$ represents the binary visibility information between candidate viewpoint set $\mathbb{V}$ and surface patch set $\mathbb{S}$.

Similar to the Traveling View Planning Problem formulated in previous work [18] but with different constraints formulation, the SCTSP problem is to find a subset $\mathbb{V}' \in \mathbb{V}$, such that the coverage constraint of the target surface is satisfied; the traveling cost of a Hamiltonian tour in $\mathbb{V}'$ and the inspection cost are minimized. The formulation is shown in Eqn. (1)-(7) below:

$$\min_{\mathbf{b},\mathbf{c}} \quad \underbrace{\sum_{i=1}^{n} \omega_i b_i}_{\text{inspection cost}} + \underbrace{\sum_{i=1}^{n} \sum_{j=1, j\neq i}^{n} c_{ij} e(v_i, v_j)}_{\text{traveling cost}} \tag{1}$$

$$\text{subject to:} \quad \sum_{i=1}^{n} b_i A_{ij} \geq 1 \quad \forall j \tag{2}$$

$$\sum_{v_i \in \mathbb{V}', v_i \neq v_j} c_{ij} = 1, \{j | \forall v_j \in \mathbb{V}'\} \tag{3}$$

$$\sum_{v_j \in \mathbb{V}', v_i \neq v_j} c_{ij} = 1, \{i | \forall v_i \in \mathbb{V}'\} \tag{4}$$

$$\sum_{v_i, v_j \in L} c_{ij} \leq |L| - 1$$

$$L \subset \mathbb{V}', |\mathbb{V}'| - 2 \geq |L| \geq 2 \tag{5}$$

$$\text{where:} \quad b_i, c_{ij} \in \{0, 1\} \tag{6}$$

$$\mathbb{V}' = \{v_i | b_i = 1, i = 1, 2, \cdots, n\}, \tag{7}$$

where w_i is the inspection cost at viewpoint v_i; (2) is the coverage constraints; (3), (4), (5) are the the constraints of DFJ formulation for TSP within $\mathbb{V}'$, adapted from [21][22]. The objective is to minimize the summation of the inspection cost and the traveling cost at the same time. In this paper, the cost is measured as the cycle time of the overall inspection task.

B. Formulation for Sequencing SCTSP

It is difficult to handle the constraints (2)-(5) directly with Integer Linear Programming method. To simplify the optimization process, we proposed a novel sequencing SCTSP formulation, and applied RKGA to solve it. The proposed sequencing SCTSP formulation is: finding a ordered viewpoint set $\mathbf{X}$, such that every element $x_i \in \mathbf{X}$ is from $\mathbb{V}$; the coverage constraint is satisfied; the sum of the inspection cost and the traveling cost is minimized.

$$\min_{\mathbf{X}} \quad \underbrace{\sum_{x_i \in \mathbf{X}} \omega_i x_i}_{\text{inspection cost}} + \underbrace{\sum_{i=1}^{n_X - 1} e(x_i, x_{i+1}) + e(x_{n_X}, x_1)}_{\text{traveling cost}} \tag{8}$$

$$\text{subject to:} \quad \sum_{i=1}^{n_X} A_{x_i, j} \geq 1 \quad \forall j, \tag{9}$$

where n_X is the size of $\mathbf{X}$, $A_{x_i,j}$ is the jth element in the row corresponding to viewpoint x_i. The visiting sequence is then directly determined by the ordered set $\mathbf{X}$, so the TSP constraints are removed from the formulation. For the coverage constraint, every surface patch is checked about the

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.

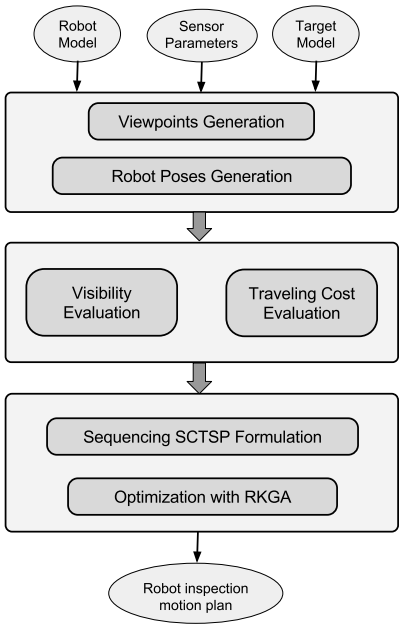


Fig. 1: The brief flowchart of the proposed method

visibility condition with the selected viewpoints, as shown in (9).

III. Proposed Method

In this paper, we propose a novel model-based, offline motion planning method for the robotic inspection applications. The proposed method first generates necessary data for the view planning and path planning problems individually, and then moves on to solve the overall SCTSP problem in a single sequencing optimization process with RKGA. The overall process is summarized in the following three steps:

1) First, randomized sampling is used to generate a set of candidate viewpoints $\mathbb{V}$ around the target object; the corresponding robot poses for each viewpoint are also generated at this stage.
2) In the second step, visibility and traveling cost are evaluated for the later optimization process:
   - A visibility matrix $\mathbf{A}$, which encodes visibility information between each surface patch on the target object and candidate viewpoint.
   - The undirected graph $\mathbf{G}(\mathbb{V}, \mathbb{E})$ with the edge set $\mathbb{E}$ that encodes the traveling costs between candidate viewpoints in the configuration space of the robot.
3) Finally, a RKGA algorithm is used to solve the formulated sequencing SCTSP problem.

As shown in Figure 1, the proposed method takes the robot model, the sensor parameters and the target object model as input, automatically generate a efficient motion plan that satisfies the inspection requirements. Further details of the proposed method are presented over the remaining part of this section.

A. Viewpoints and Robot Poses Generation

The first step of the proposed method is to generate a set of candidate viewpoints $\mathbb{V}$ about the target object. For each generated viewpoint, a corresponding robot pose is also generated via inverse kinematics.

1) Sampling Viewpoints: In this paper, a randomized sampling method is used to generate the positions of the candidate viewpoints. The sampling process of the viewpoints is performed in Euclidean space around the target object, within the maximum viewing range of the sensor. Redundant viewpoints are generated in this step.

In addition to the position, the viewing direction is chosen by potential field method [15]. This method ensures each viewpoint is oriented in a direction that captures a weighted average of nearby surface patches on the target object. The viewing direction $\mathbf{v}$ of a given viewpoint located at $\mathbf{p_{vp}}$ is:

$$\mathbf{v} = \frac{\sum_i^N \frac{K\mathbf{d_i}}{\|\mathbf{d_i}\|^3}}{\| \sum_i^N \frac{K\mathbf{d_i}}{\|\mathbf{d_i}\|^3} \|}, \tag{10}$$

$$\text{where} \quad \mathbf{d_i} = \mathbf{p_{vp}} - \mathbf{p_{patch_i}}$$

$$\text{for all} \quad \{\mathbf{p_{patch_i}} | (\mathbf{p_{vp}} - \mathbf{p_{patch_i}})^T (\mathbf{p_{vp}} - \mathbf{p_{patch_i}}) < d_{max}\},$$

where K is a constant value; $\mathbf{p_{patch_i}}$ is the position of ith surface patch; d_{max} defines the maximum inclusion distance, only surface patches located within this distance to the viewpoint will be included in the calculation of the mean viewing direction.

2) Generating Robot Poses for each Viewpoint: A valid robot pose must be defined for each member in the candidate viewpoint set. To do this, each candidate viewpoint is input into the IKFast [23] inverse kinematics algorithm to generate a corresponding robot pose. The Flexible Collision Library (FCL) [24] is then used to test the robot pose for collision with the surrounding environment. If either of these tasks fails, we then rotate the viewpoint about its optical axis until a valid robot pose is found. We are able to do this, as rotating a given viewpoint about its optical axis does not significantly alter the sensors ability to capture the geometric data of a given scene. The subsequent rotated viewpoint, and if it is a valid robot pose, then replaces its former counterparts in the candidate viewpoint set. If no valid pose for a given viewpoint is possible, the viewpoint is removed from the candidate viewpoint set.

B. Evaluation of Viewpoints and Robot Poses

In the second step, two properties from each candidate viewpoint are evaluated. First, a measure of visibility between each surface patch on the target object and each candidate viewpoint is calculated and stored in a two-dimensional visibility matrix $\mathbf{A}$. Then, the edges of the Graph $\mathbf{G}(\mathbb{V}, \mathbb{E})$ are found by evaluating the associated traveling costs for the robot to travel between the candidate viewpoints.

1) Evaluation of Visibility: With the 6-DOF position and orientation of the viewpoints, the visibility of the surface patches can be then evaluated. A triangular surface patch is considered visible from a given viewpoint if all the following conditions are met for all of its vertices:

- The vertices of the surface patch must be in the Field-Of-View (FOV) of viewpoint.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.

- The vertices of the surface patch must be in the Field-Of-Depth (FOD) of viewpoint.
- The viewing angle must be within the specified range.
- There is no occlusion in between the viewpoint and surface patch.

The visibility criterion is adapted from our previous work [15], and is similar to the methods documented in [17] and [11]. The visibility evaluation process generates a $n \times m$ binary visibility matrix $\mathbf{A}$, where n is the number of candidate viewpoints, and m is the number of surface patches on the target model. The information encoded in this matrix indicates whether a given surface patch is visible to a particular viewpoint. For example, with the value of A_{ij}, 1 indicates the surface patch j is visible to the viewpoint i, whereas 0 means it is not visible.

2) Evaluation of Traveling Costs: The required collision-free robot path between the robot poses at two given viewpoints is computed using the MoveIt [25] framework from Robot Operating System (ROS) [26]. The framework uses the Open Motion Planning Library (OMPL) [27] for path planning and FCL [24] for collision detection. The local planner used in this paper is the Rapidly exploring Random Tree connect (RRT-connect) algorithm [28], similar to the adaptation in the previous work [7]. RRT-connect is a variation of RRT, and operates by generating poses randomly in the robot's configuration space and initiates exploration from both starting and goal poses. The method is considered probabilistically complete [29], and has been used in this work as it is able to generate high quality robot paths between poses in a short amount of time. Note that in the proposed framework, other local planner such as RRT* can also be used to generate the pose-to-pose paths.

The cubic spline is used to parametrize the generated robot paths with speed and acceleration data of the robot's actuators. This allows the robot paths to be converted into the robot trajectories. The subsequent traveling times can then be computed. The traveling costs (time) between the robot poses at the viewpoints are calculated and stored in the corresponding edges of $\mathbf{G}(\mathbb{V}, \mathbb{E})$.

C. Random-key Genetic Algorithm for SCTSP Problem

With the visibility matrix and $\mathbf{G}(\mathbb{V}, \mathbb{E})$ generated in the previous sections, the SCTSP is now formulated as a variation of sequencing problem. A Random-Key Genetic Algorithm (RKGA) is then applied to solve it in a single optimization process.

1) The Random-key Genetic Algorithm: The RKGA method is a popular technique used in many combinatorial optimization problems, such as the TSP problem [30] and other sequencing problems [31]. Similar to standard GA method, in RKGA the population consists of many chromosomes; each chromosome featuring a number of genes. Different from the traditional GAs, the RKGA makes use of random real numbers as the key stored in each gene of the chromosome. A decoder then deciphers information in the genes of the chromosome, before evaluating the fitness/cost score.

In this paper, we use real number range from 0 to 1 as the random key to encode the information. The decoding strategy first sorts the chromosome by the random key value stored in the genes; then instead of using all sorted genes for fitness evaluation in the chromosome in literature [31], the proposed method continually adds the sorted genes to solution set and evaluate the fitness until the coverage constraint is satisfied. This approach bypasses the coverage constraint by evaluating coverage criteria in the fitness function of RKGA, as shown in Algo. 1. For example, a chromosome with the genes $(0.31, 0.25, 0.84, 0.64, 0.93)$ is first decoded as a sequence of $2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 5$, then during the fitness evaluation, the viewpoint with a index number 2 will firstly be added to the solution set, the cost will be evaluated accordingly. Meanwhile the fitness function will evaluate the coverage constraint as well; if the coverage constraint is not satisfied, the next viewpoint with index number 1 will be added to the solution set, this process repeatedly add n_x viewpoints to the solution set until the coverage constraint is satisfied.

The encoding/decoding strategy used in this paper has the advantages of not dealing with the constraints directly, which simplifies the optimization process. For example, if we use the Eqn. (1) - (7) as a Integer Linear Programming (ILP) problem, the coverage constraint (2) and TSP subtour elimination constraints (3)-(5) must be handled by the optimization algorithm. In addition, note that though written in a compact form, the coverage constraint (2) actually have lots of linear equations with integer components.

2) Solving the Sequencing SCTSP using RKGA: With the sequencing formulation of the SCTSP problem at Sec. II, the visibility matrix and $\mathbf{G}(\mathbb{V}, \mathbb{E})$, a optimization formulation with adaptation of RKGA is shown in Eqn. 11. In this paper, we assume constant inspection time ω_0 for each viewpoint; and the set covering constraint (13) is relaxed such that a certain coverage ratio δ is desired instead of full coverage. Additionally, as mentioned in Section II, since the proposed method is designed for production line, the robot should start at its home pose, and return to the home pose at after finishing the inspection task. The formulation is shown below:

$$\min_{\mathbf{x}', n_x} \quad \underbrace{\omega_0 n_x}_{\text{inspection cost}} + \underbrace{\sum_{i=1}^{n_x - 1} e(x'_i, x'_{i+1}) + e(x'_h, x'_1) + e(x'_{n_x}, x'_h)}_{\text{traveling cost}} \tag{11}$$

$$\text{where} \quad x'_i \in \mathbb{V} \tag{12}$$

$$\text{subject to} \quad \sum_{i=1}^{n_x} ((\sum_{j \in \mathbf{x}'} \mathbf{A_j}) \geq 1) \geq \delta * m, \tag{13}$$

where $\mathbf{x}'$ is the ordered set of the selected viewpoints; $e(x'_i, x'_{i+1}) \in \mathbb{E}$ is the robot traveling cost between its poses at viewpoints x'_i and x'_{i+1}; x'_h represents the robot home pose, so $e(x'_h, x'_1)$ means the traveling cost between the

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.

home pose and the pose at first viewpoint in the solution, $e(x'_{n_x}, x'_h)$ means the traveling cost between the home pose and the pose at last viewpoint in the solution; (13) is the coverage constraint; $\mathbf{A}$ is the $n \times m$ visibility matrix; $\mathbf{A_j}$ is the jth row of matrix $\mathbf{A}$; δ is the required coverage ratio. In this formulation, we slightly adjust the SCP so that a certain coverage ratio instead of full coverage is required, which is similar to the formulation reported in literature[32]. In RKGA, the chromosome $\mathbf{x}$ is a list with random key x_i stored in its genes, the ordered viewpoints set $\mathbf{x}'$ is obtained from the chromosome $\mathbf{x}$; the coverage constraint (13) is evaluated in the fitness function of RKGA.

Algorithm 1 Evaluating the Cost of a Given Chromosome

Input: $\mathbf{x}$: the chromosome with the random keys stored in its genes; $\mathbf{G}(\mathbb{V}, \mathbb{E})$: the graph; $\mathbf{A}$: the visibility matrix; δ: the desired coverage ratio; ω_0: inspection cost at viewpoint.
Output: y: the fitness of the input chromosome; n_x: the number of first few viewpoints needed for the required coverage;

```
y = ω_0
n_x = 0
x' ← decodeBySort(x)
y = y + e(x'_h, x'_1)
for each x'_i ∈ x' and i > 1 do
    y = y + ω_0 + e(x'_i, x'_{i+1})
    n_x = n_x + 1
    δ' ← findCoverageRatio(A, x'_i)
    if δ' > δ then
        break
    end if
end for
y = y + e(x'_{n_x}, x'_h)
return y, n_x
```

In the proposed approach, the number of selected viewpoints n_x is not directly encoded in the chromosome. Instead, n_x is determined by the algorithm's fitness function, our approach only chooses the first n_x viewpoints in the ordered set $\mathbf{x}'$ that satisfy the required coverage constraints. By doing this, all offspring generated will be able to generate feasible solutions, and we can then bypass the coverage constraint introduced by the SCP, as well as the TSP subtour elimination constraints. The details of cost function that evaluates the cost of a given chromosome are shown in Algo. 1, where "decodeBySort($\mathbf{x}$)" returns a sorted list by sorting the value of $\mathbf{x}$.

IV. Simulation, Experiment and Discussion

TABLE I: Sensor parameters

FOV	$30°, 21°$
Camera Pixels	536×371
Viewing Angle	$75°$
FOD (meters)	$(0.40, 1.00)$
Coverage ratio	$\delta = 99.5\%$

In this paper, the validity and effectiveness of the proposed method is verified through both simulation and experiment. The performance of the proposed method is compared to the more traditional approaches where the SCP and TSP are solved individually. Two other approaches are used in these comparative tests: The first approach solves both SCP and TSP using two separate GAs (labelled as GA-SCP-GA-TSP); the other method solves the SCP using a greedy method [15] and then solves the TSP using a RKGA solver (labelled as Greedy-SCP-GA-TSP). In this paper, we assume that the inspection time at different viewpoints carries the same cost, making the total inspection cost directly proportional to the number of required viewpoints, the assumption is reasonable because the inspection time is only related to the 3D scanner, the 3D reconstruction of the polygonal model is done in post-processing. Traveling costs are lifted from the edge set $\mathbb{E}$ of $\mathbf{G}(\mathbb{V}, \mathbb{E})$ generated in the earlier process.

In both simulation and experiment, the robot used is an ABB-IRB-4600 6-DOF industrial manipulator. An Artec Eva 3D scanner is mounted on the end-effector of the robot. According to its data sheet, the scanner has a FOD of $0.40m$ to $1.00m$, as well as a FOV of $30°$ and $21°$ in the horizontal and vertical directions, respectively. In this paper, the sensor parameters used for planning are listed in Table I.

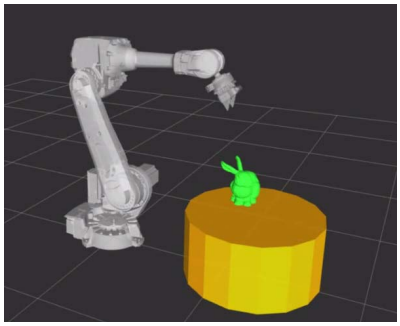
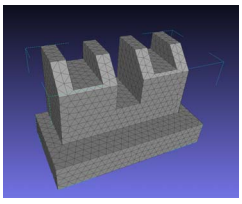
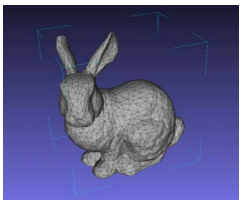


Fig. 2: The simulation setup: a target object (in green) is placed on a machining table (in orange) for inspection; the ABB IRB-4600 industrial manipulator (in grey) has a 3D scanner mounted on the robot end-effector. The pose displayed in the figure is used as home pose for this paper



(a) Target object 1: the bounding box size is $0.24 \times 0.47m \times 0.30m$



(b) Target object 2: the bounding box size is $0.20m \times 0.30m \times 0.30m$

Fig. 3: The target objects

A. Setup

The simulation-based tests performed in this work were carried out using ROS [26]. ROS provides an built-in visualization environment (Rviz), planning interface (MoveIt), motion planning library (OMPL [27]) and collision detection (FCL [24]). Inverse kinematic queries were solved using IKFast [23]. Note that though multiple inverse kinematics solutions may exist for certain poses, we do not address this issue in the paper. The multiple solution problem of inverse kinematics can be addressed by a sampling strategy.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.

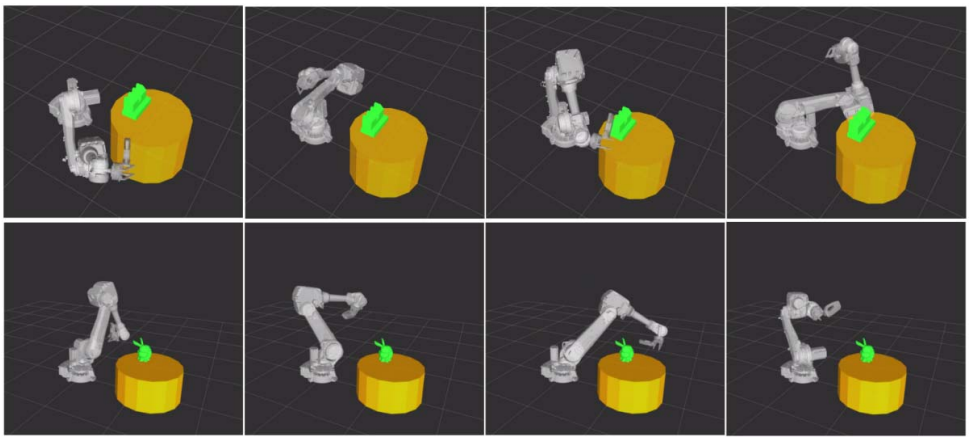


Fig. 4: Example of robot poses to position the 3D scanner to desired viewpoints for inspecting target object 1 and 2

In the simulation environment, a machining table is placed in front of the industrial manipulator, as shown in Figure 2. The table is modelled as cylinder to speed up collision detection processing, while the bounding box of the target object is used for collision check with the target object. Two simulation tests, each featuring different geometric shapes, shown in Figure 3, were performed. The respective target object is placed on the circular table 0.7m in height for the first test and 0.5m for the second test; the target object is positioned 0.9m away from the robot base location, as shown in Figure 2. A constant inspection time of 1 second at each viewpoint was used.

B. Computational Results

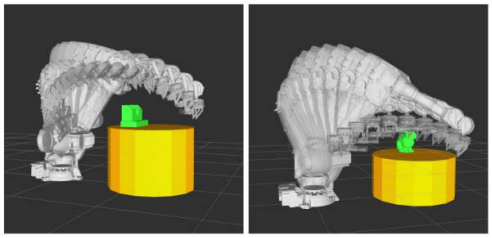


Fig. 5: The example trajectory for inspecting both target objects

The results for the two simulations are presented in Table II, III and Figure 6, where T_{avg} is the average cycling time based on ten trials, T_{std} is the standard variation of the cycling time, and N_{vp} is the average number of viewpoints required for the inspection. In Figure 6a and 6b, different numbers of candidate viewpoints are used in the comparison; the same set of sampled candidate viewpoints are used for the three different methods. Note that for Greedy-SCP-GA-TSP method, because the greedy method finds deterministic results of viewpoints, and the scale of the TSP problem within the resultant viewpoints is small, the results for Greedy-SCP-GA-TSP have very little variations.

The results of the first simulation indicate that the proposed method (labeled as GA-SCTSP) is able to generate a better solution for the given inspection task, as shown in Table II. For the overall cycle time, respective reductions of 10.8% and 19.6% in time were recorded compared to the previous methods. Example poses and robotic trajectories are shown in Figure 4 and Figure 5. Figure 6a also shows the results of different methods and different number of viewpoints generated.

In the second test, the advantage of the proposed method was observed to improve further. In terms of number of required viewpoints, the GA-SCP-GA-TSP method performs best, requiring fewer viewpoints than the GA-SCTSP and Greedy-SCP-GA-TSP methods. However in terms of overall time costs, the proposed method required 18.6% and 23.3% less cycling time compared to the previous methods on average, as shown in Table III. A 28.4% overall time reduction compared to Greedy-SCP-GA-TSP methods is recorded when using 200 sampled viewpoints. Example poses and robotic trajectories are shown in Figure 4 and 5.

TABLE II: Results for inspecting target object 1

	200 samples			**300 samples**			**400 samples**		
	T_{avg}	T_{std}	N_{vp}	T_{avg}	T_{std}	N_{vp}	T_{avg}	T_{std}	N_{vp}
GA-SCP-GA-TSP	35.1	1.50	9.2	34.9	1.40	9.3	34.8	1.61	9.1
Greedy-SCP-GA-TSP	43.6	0.00	12.0	38.0	0.00	11.0	35.5	0.00	10.0
GA-SCTSP	31.3	0.62	9.2	31.1	0.91	9.3	30.8	0.71	8.9

TABLE III: Results for inspecting target object 2

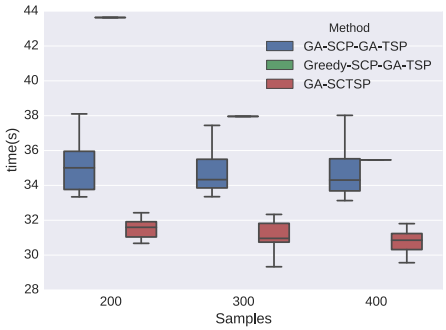
	200 samples			**300 samples**			**400 samples**		
	T_{avg}	T_{std}	N_{vp}	T_{avg}	T_{std}	N_{vp}	T_{avg}	T_{std}	N_{vp}
GA-SCP-GA-TSP	37.1	2.17	8.3	34.2	1.25	8.5	35.2	1.51	7.4
Greedy-SCP-GA-TSP	43.2	0.00	11.0	33.7	0.00	9.0	37.0	0.02	8.0
GA-SCTSP	30.9	0.53	9.2	28.4	1.57	9.1	27.5	1.46	8.7

C. Experiment

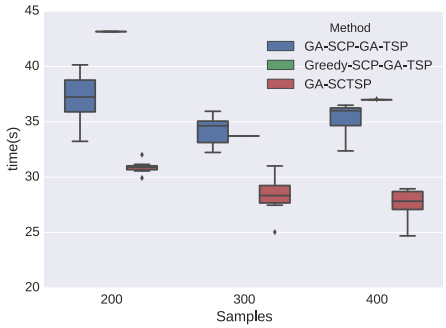
To further verify the effectiveness of the proposed method in a real world situation, we conducted an experimental test using an ABB-IRB-4600 industrial robot and an Artec Eva 3D scanner. The experiment setup is shown in Figure 7. A square table of 0.51m in height is placed in front of the robot and the target object is placed 0.75m away from the robot.

To perform the planned robot motions with the real world robotic hardware, the ROS-industrial package [33] is used, which allows the trajectories generated in the simulation environment to be sent to the real-world robot controller

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.



(a) Targat Object 1



(b) Targat Object 2

Fig. 6: Total cycling time required of different methods for inspecting target objects (based on ten trials)

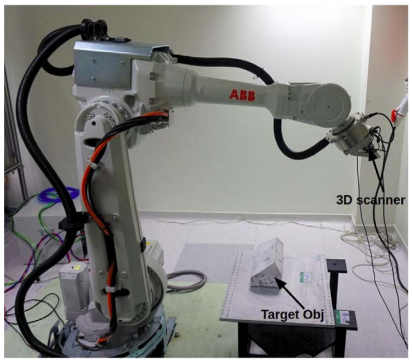
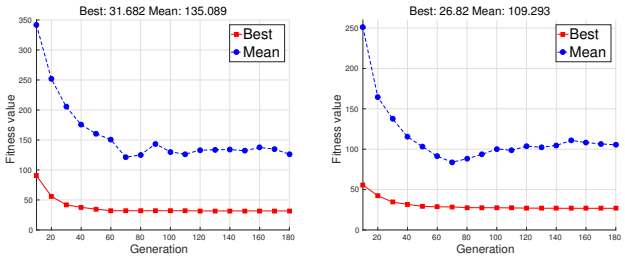


Fig. 7: Experimental setup

via Ethernet directly. These trajectory commands control the robot so it moves to the planned viewpoints, and the scanner is configured via I/O signals to perform scanning operations at the required moments. Example planned poses and their corresponding actual poses are shown in Figure 8. Once all required scans were captured, the Artec SDK library is used, along with robot pose feedback data, to stitch together an accurate composite CAD model of the target object, as shown in Figure 9. During the experiment, the inspection plan is computed using 400 samples of viewpoints. With the proposed method, 19.2 seconds are required for the inspection plan, while GA-SCP-GA-TSP requires 23.8 seconds and Greedy-SCP-GA-TSP requires 25.7 seconds for the task.

D. Covergence and Computational Cost



(a) Targat Object 1 (b) Targat Object 2

Fig. 10: Convergence of RKGA

As shown in Figure 10, it is observed that the results of RKGA start to converge before 100 generations. 400 samples are used in the figure. The score of the best known chromosome is shown in red, while the average score is shown in blue.

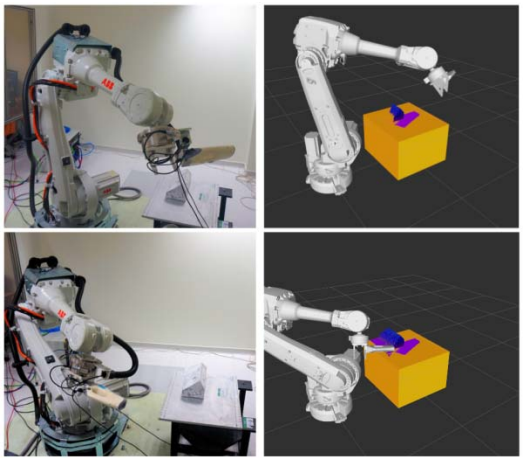


Fig. 8: Visualization of example inspection poses during the experiment and partial scan in Rviz (right) and example actual robot poses (left) for inspection during the experiment.

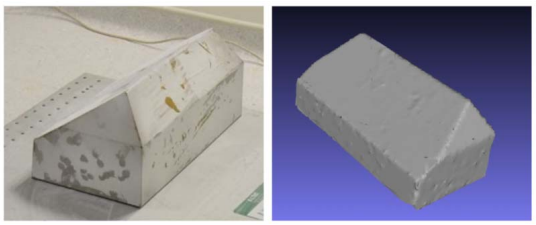


Fig. 9: The photo and 3D polygonal recontruction model of the target object

The runtime of 400 samples with the proposed method is about 3 hours on a Intel i7-5500U laptop CPU with 12G RAM. The computation of the pose-to-pose trajectories and traveling costs takes most of the computational time.

E. Discussion

Both the simulation and experiment show that integrating the SCP and TSP components of the generalized planning process together can provide a better solution than solving the SCP and TSP separately. This can be attributed to the fact that when the problems are solved separately, the SCP

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.

solver is not aware of the traveling cost, which results in sub-optimal solutions. By integrating these two problems during the optimization process, the solver will continue to search for a minimized overall cost even after the SCP solution has already been solved.

V. Conclusion

In this paper, we propose a novel planning method for industrial inspection applications by addressing the view planning and path planning problems simultaneously via a sequencing SCTSP formulation. The proposed approach automatically generates time-efficient, collision-free motion plan that satisfies the requirement of the robotic inspection task. The effectiveness of the proposed method was demonstrated via computational and experimental tests, where it showed good improvement over existing methods. In these tests, the method was observed to generate robot paths with an improvement of up to 28.4% in required cycle time. This indicates that the method would prove useful in generating paths for the inspection tasks on an automated production line, where the repetitive nature of the task demands a short cycle time for such operations. An experiment with an ABB robotic inspection system was also conducted to validate the overall approach.

The future work will consider the multiple IK solutions and kinematic redundant robot in the motion planning. In addition, extending the current planning algorithm to handle object with non-fixed poses is also a promising research direction.

References

[1] Q. Wu, J. Lu, W. Zou, and D. Xu, "Path planning for surface inspection on a robot-based scanning system," in *IEEE International Conference on Mechatronics and Automation*. IEEE, 2015, pp. 2284–2289.

[2] R. Raffaeli, M. Mengoni, M. Germani, and F. Mandorli, "Off-line view planning for the inspection of mechanical parts," *International Journal on Interactive Design and Manufacturing (IJIDeM)*, vol. 7, no. 1, pp. 1–12, 2013.

[3] C. N. Macleod, G. Dobie, S. G. Pierce, R. Summan, and M. Morozov, "Machining-based coverage path planning for automated structural inspection," *IEEE Transactions on Automation Science and Engineering*, 2016.

[4] K. A. Tarabanis, R. Y. Tsai, and P. K. Allen, "The mvp sensor planning system for robotic vision tasks," *IEEE Transactions on Robotics and Automation*, vol. 11, no. 1, pp. 72–85, 1995.

[5] W. Scott, G. Roth, and J.-F. Rivest, "View planning for automated 3d object reconstruction inspection," *ACM Computing Surveys*, vol. 35, no. 1, 2003.

[6] S. Chen, Y. Li, and N. M. Kwok, "Active vision in robotic systems: A survey of recent developments," *The International Journal of Robotics Research*, vol. 30, no. 11, pp. 1343–1377, 2011.

[7] I. Gentilini, K. Nagamatsu, and K. Shimada, "Cycle time based multi-goal path optimization for redundant robotic systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2013, pp. 1786–1792.

[8] S. Alatartsev, S. Stellmacher, and F. Ortmeier, "Robotic task sequencing problem: A survey," *Journal of Intelligent & Robotic Systems*, vol. 80, no. 2, p. 279, 2015.

[9] A. Bircher, K. Alexis, M. Burri, P. Oettershagen, S. Omari, T. Mantel, and R. Siegwart, "Structural inspection path planning via iterative viewpoint resampling with application to aerial robotics," in *IEEE International Conference on Robotics and Automation*. IEEE, 2015, pp. 6423–6430.

[10] B. Englot and F. S. Hover, "Three-dimensional coverage planning for an underwater inspection robot," *The International Journal of Robotics Research*, vol. 32, no. 9-10, pp. 1048–1073, 2013.

[11] W. R. Scott, "Model-based view planning," *Machine Vision and Applications*, vol. 20, no. 1, pp. 47–69, 2009.

[12] S. Isler, R. Sabzevari, J. Delmerico, and D. Scaramuzza, "An information gain formulation for active volumetric 3d reconstruction," in *2016 IEEE International Conference on Robotics and Automation*. IEEE, 2016, pp. 3477–3484.

[13] C. Potthast and G. S. Sukhatme, "A probabilistic framework for next best view estimation in a cluttered environment," *Journal of Visual Communication and Image Representation*, vol. 25, no. 1, pp. 148–164, 2014.

[14] A. Mavrinac, X. Chen, and J. L. Alarcon-Herrera, "Semiautomatic model-based view planning for active triangulation 3-d inspection systems," *IEEE/ASME Transactions on Mechatronics*, vol. 20, no. 2, pp. 799–811, 2015.

[15] W. Jing, J. Polden, W. Lin, and K. Shimada, "Sampling-based view planning for 3d visual coverage task with unmanned aerial vehicle," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2016, pp. 1808–1815.

[16] I. Gentilini, "Multi-Goal Path Optimization for Robotic Systems with Redundancy based on the Traveling Salesman Problem with Neighborhoods," Ph.D. dissertation, Carnegie Mellon University, May 2012.

[17] S. Chen and Y. Li, "Automatic sensor placement for model-based robot vision," *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, vol. 34, no. 1, pp. 393–408, 2004.

[18] P. Wang, R. Krishnamurti, and K. Gupta, "View planning problem with combined view and traveling cost," in *IEEE International Conference on Robotics and Automation*. IEEE, 2007, pp. 711–716.

[19] P. Wang, K. Gupta, and R. Krishnamurti, "Some complexity results for metric view planning problem with traveling cost and visibility range," *IEEE Transactions on Automation Science and Engineering*, vol. 8, no. 3, pp. 654–659, 2011.

[20] B. J. Englot, "Sampling-based coverage path planning for complex 3d structures," Ph.D. dissertation, Massachusetts Institute of Technology, 2012.

[21] G. Dantzig, R. Fulkerson, and S. Johnson, "Solution of a large-scale traveling-salesman problem," *Journal of the operations research society of America*, vol. 2, no. 4, pp. 393–410, 1954.

[22] G. Laporte, "The traveling salesman problem: An overview of exact and approximate algorithms," *European Journal of Operational Research*, vol. 59, no. 2, pp. 231–247, 1992.

[23] R. Diankov and J. Kuffner, "Openrave: A planning architecture for autonomous robotics," *Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-08-34*, vol. 79, 2008.

[24] J. Pan, S. Chitta, and D. Manocha, "Fcl: A general purpose library for collision and proximity queries," in *IEEE International Conference on Robotics and Automation*. IEEE, 2012, pp. 3859–3866.

[25] I. A. Sucan and S. Chitta, "Moveit!" *Online Availabl e: http://moveit. ros. org*, 2013.

[26] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: an open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.

[27] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, http://ompl.kavrakilab.org.

[28] J. J. Kuffner and S. M. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *IEEE International Conference on Robotics and Automation*, vol. 2. IEEE, 2000, pp. 995–1001.

[29] S. M. LaValle, *Planning algorithms*. Cambridge university press, 2006.

[30] L. V. Snyder and M. S. Daskin, "A random-key genetic algorithm for the generalized traveling salesman problem," *European Journal of Operational Research*, vol. 174, no. 1, pp. 38–53, 2006.

[31] J. C. Bean, "Genetic algorithms and random keys for sequencing and optimization," *ORSA journal on computing*, vol. 6, no. 2, pp. 154–160, 1994.

[32] W. Jing, J. Polden, P. Y. Tao, W. Lin, and K. Shimada, "View planning for 3d shape reconstruction of buildings with unmanned aerial vehicles," in *International Conference on Control, Automation, Robotics and Vision*. IEEE, 2016.

[33] S. Edwards and C. Lewis, "Ros-industrial: applying the robot operating system (ros) to industrial applications," in *IEEE International Conference on Robotics and Automation, ECHORD Workshop*, 2012.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on January 25,2025 at 03:17:51 UTC from IEEE Xplore. Restrictions apply.